Deploying Your Personal Website with Docker

A Hands-On Workshop

SUNY New Paltz — Computer Science Lab

Platform: Hydra Cluster (`hydra.newpaltz.edu`)
Project: PersonalSite — React + TypeScript Portfolio
Stack: React, TypeScript, Docker, Nginx, Traefik

By the end of this workshop, you will have your own personal portfolio website running in a Docker container, accessible on the public internet at `hydra.newpaltz.edu/students/`*`you`*`/mysite/`

Spring 2026

Contents

1 Introduction **2**

2 Technology Overview **2**
- 2.1 React . . . 3
- 2.2 TypeScript . . . 4
- 2.3 SCSS (Sass) . . . 4
- 2.4 Node.js and npm . . . 5
- 2.5 Docker . . . 6
  - 2.5.1 The Dockerfile — Multi-Stage Build . . . 6
- 2.6 Docker Compose . . . 7
- 2.7 Nginx . . . 8
- 2.8 Traefik (Reverse Proxy) . . . 8

3 Project Architecture **9**
- 3.1 Project Structure . . . 9
- 3.2 How the Pieces Fit Together . . . 10

4 Workshop: Deploy Your Site **10**
- 4.1 Step 1: Connect to Your Container . . . 10
- 4.2 Step 2: Clone the Repository . . . 10
- 4.3 Step 3: Personalize Your Site . . . 11
  - 4.3.1 3a. Your name, title, and links . . . 11
  - 4.3.2 3b. Your email . . . 11
  - 4.3.3 3c. HTML metadata . . . 11
  - 4.3.4 3d. Footer credit . . . 11
  - 4.3.5 3e. Your resume (optional) . . . 12
  - 4.3.6 3f. Set relative asset paths . . . 12
- 4.4 Step 4: Build the Docker Image . . . 12
- 4.5 Step 5: Run the Container . . . 13
- 4.6 Step 6: Expose Your Site to the Internet . . . 13
- 4.7 Step 7: Make It Persistent with Supervisord . . . 14

5 Common Operations **14**
- 5.1 Updating Your Site . . . 14
- 5.2 Viewing Logs . . . 15
- 5.3 Debugging . . . 15
- 5.4 Removing Your Site . . . 15

6 Key Concepts Recap **15**

7 Appendix: The Full Request Lifecycle **16**

8 Appendix: Troubleshooting **16**

9 Appendix: Going Further **17**

1 Introduction

This workshop walks you through deploying a real web application from source code to a live, publicly accessible URL. Along the way, you will learn what each technology in the stack does and why professional developers choose it.

The project is a personal portfolio site built with **React** and **TypeScript**. It features an animated particle background, dark/light theme toggle, social media links, and an embedded resume viewer. The source code is at:

`https://github.com/ndg8743/PersonalSite`

You will:

1. Clone the repository into your Hydra container.
2. Personalize the site with your own name, links, and resume.
3. Build a production Docker image using a multi-stage `Dockerfile`.
4. Run the container with Nginx serving your compiled site.
5. Expose it to the internet through Hydra's reverse proxy.

Prerequisites

You need a working Hydra student container with:

- SSH or VSCode access (`hydra.newpaltz.edu/students/`*`you`*`/vscode/`)
- Node.js (v20) and npm (pre-installed)
- Docker (available via the Docker-in-Docker sidecar)

2 Technology Overview

Before touching any code, let's understand *what* each tool does and *why* it exists. The diagram below shows how a request travels from a user's browser to your running site:

```
Browser (user visits your URL)
   |
   v
Traefik (reverse proxy - routes by URL path)
   |
   v
Nginx (web server - serves static files)
   |
   v
/usr/share/nginx/html/  (your compiled React app)
   ^
   |  (built at image-build time by)
   |
Node.js + npm (compiles React/TypeScript into HTML/CSS/JS)
   ^
   |  (your source code, written in)
   |
React + TypeScript + SCSS
```

2.1 React

What it is: A JavaScript library (created by Facebook/Meta) for building user interfaces out of reusable *components*.

Why it matters: Instead of writing raw HTML and manually updating the page when data changes, React lets you describe *what* the UI should look like and handles all the DOM manipulation for you. Each piece of UI is a function that returns JSX (HTML-like syntax inside JavaScript):

Listing 1: src/components/Content.tsx — A React component

```
export const Content = () => {
  const { config, theme } = useContext(AppContext);

  return (
    <>
      <C.Name $theme={theme}>{config.name.display}</C.Name>
      <C.Title $theme={theme}>{config.title.display}</C.Title>
    </>
  );
};
```

Where it is in this project:

- `src/App/App.tsx` — Root component, assembles the page layout.
- `src/components/` — Individual UI pieces (Content, Buttons, Toggle, Footer, Particles, PdfViewer).
- `src/App/config.tsx` — Your name, title, and social links.
- `src/App/AppContext.tsx` — Shared state (theme, config) using React Context.

Key concept — Component tree: React apps are a tree of components. In this project:

Listing 2: Component hierarchy

```
<App>
  <AppProvider>        <!-- Theme and config state -->
    <Toggle />         <!-- Dark/light mode switch -->
    <Content />        <!-- Your name and title -->
    <Buttons />        <!-- GitHub, LinkedIn, Resume, Email -->
    <Footer />         <!-- "Designed and built by..." -->
    <Particles />      <!-- Animated background -->
  </AppProvider>
</App>
```

2.2 TypeScript

What it is: A superset of JavaScript that adds *static types*. Every `.ts` and `.tsx` file is TypeScript.

Why it matters: JavaScript lets you pass anything anywhere — a string where a number is expected, a missing field, a misspelled property name. These bugs only show up when the code runs (often in production). TypeScript catches them *at compile time*:

Listing 3: src/types/config.interface.ts — Type safety

```
export interface Config {
  buttons: Button[];  // Must be an array of Button objects
  name: Content;      // Must have a ‘display‘ string
  title: Content;
}

export interface Button extends Content {
  ariaLabel: string;
  href: string;
  icon: JSX.Element;
  name: string;
}
```

If you try to add a button without an `ariaLabel`, TypeScript shows a red underline in your editor *before* you even run the code.

Where it is: The `tsconfig.json` in the project root configures the TypeScript compiler. Every `.tsx` file is a React component written in TypeScript.

2.3 SCSS (Sass)

What it is: A CSS preprocessor that adds variables, nesting, and mixins to standard CSS. Files use the `.scss` extension.

Why it matters: Plain CSS gets repetitive in large projects. SCSS lets you write cleaner stylesheets. This project also uses **styled-components** (CSS-in-JS) for most styling, but the global layout is in SCSS:

Listing 4: src/App/App.scss — Global layout

```
.app {
```

```
  width: 100vw;
  height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}
```

styled-components is used for theme-aware styling. Instead of separate CSS files, styles live right next to the component logic:

Listing 5: Theme-aware styled component

```
const Name = styled.h1<{ $theme: Theme }>`
  color: ${({ $theme }) => $theme.primaryTextColor};
  font-size: 6rem;
  transition: color 0.5s linear;
`;
```

2.4 Node.js and npm

What Node.js is: A runtime that lets you execute JavaScript outside the browser. It powers the build tools that compile your React code into plain HTML, CSS, and JS files.

What npm is: The Node Package Manager. It installs third-party libraries (React, TypeScript, styled-components, etc.) listed in `package.json`.

Key commands:

Command	What it does
`npm ci`	Clean install — installs exact versions from `package-lock.json`. Deterministic and fast.
`npm start`	Starts a development server with hot reload (changes appear instantly).
`npm run build`	Compiles and optimizes everything into a `build/` folder of static files.
`npm test`	Runs the test suite.

Where it is:

- `package.json` — Lists all dependencies and scripts.
- `package-lock.json` — Locks exact versions for reproducibility.
- `.nvmrc` — Specifies the Node.js version (`nvm use`).

Build vs. Runtime

Node.js is only needed to *build* the site. Once built, the output is plain HTML/CSS/JS that any web server can serve. That's why the final Docker image uses Nginx instead of Node.js — it's 10x smaller and faster.

2.5 Docker

What it is: A platform for packaging applications into *containers* — lightweight, isolated environments that include everything needed to run: code, runtime, libraries, and system tools.

Why it matters: "It works on my machine" is the bane of software engineering. Docker eliminates this by guaranteeing that if a container works on your laptop, it works identically on any server. Every dependency is captured in the image.

Key concepts:

Term	Meaning
Image	A read-only template. Think of it as a snapshot of a configured machine. Built from a `Dockerfile`.
Container	A running instance of an image. You can start, stop, and delete containers without affecting the image.
Dockerfile	A recipe (text file) that tells Docker how to build an image, step by step.
Layer	Each instruction in a Dockerfile creates a layer. Docker caches layers, so unchanged steps are instant on rebuild.
Registry	A repository for images (like GitHub for code). Docker Hub is the default public registry.

2.5.1 The Dockerfile — Multi-Stage Build

This project uses a **multi-stage build**. This is a key Docker pattern: use one image to *build* your app, then copy only the output into a smaller image that *serves* it. The Node.js tools, source code, and `node_modules` are discarded — only the compiled HTML/CSS/JS makes it into the final image.

Listing 6: Dockerfile — annotated

```
# ---- Stage 1: BUILD ----
# Start with Node.js to compile our React app
FROM node:18-alpine AS build
WORKDIR /app

# Copy package files FIRST (Docker caches this layer)
COPY package*.json ./
RUN npm ci --ignore-scripts

# Copy source and build
COPY . .
RUN npm run build
# Output: /app/build/ (static HTML, CSS, JS)

# ---- Stage 2: SERVE ----
# Switch to a tiny Nginx image (no Node.js needed!)
FROM nginx:alpine

# Copy ONLY the compiled output from Stage 1
COPY --from=build /app/build /usr/share/nginx/html

# Copy our custom Nginx config
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

```

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Why Alpine?

Both `node:18-alpine` and `nginx:alpine` use Alpine Linux, a minimal distribution that's only 5 MB. A full Ubuntu-based Node image is 900+ MB; the Alpine variant is ~180 MB. The final Nginx image is ~40 MB. This means faster pulls, less disk, and a smaller attack surface.

Why copy `package*.json` first?

Docker caches each layer. If you copy all files first and then run `npm ci`, changing *any* source file invalidates the cache and reinstalls all dependencies (slow). By copying only `package.json` first, the `npm ci` layer is cached and only re-runs when dependencies actually change. This is called **layer caching optimization**.

2.6 Docker Compose

What it is: A tool for defining multi-container applications in a single YAML file (`docker-compose.yml`). Instead of typing long `docker run` commands, you describe everything declaratively.

Why it matters: Real applications have multiple services (web server, database, cache, reverse proxy). Compose lets you start everything with one command and configure networking, volumes, resource limits, and health checks in one place.

Listing 7: docker-compose.yml — annotated (simplified)

```yaml
services:
  personal-site:
    build: .                        # Build from Dockerfile
    container_name: personal-site
    restart: unless-stopped         # Auto-restart on crash

    # Security hardening
    security_opt:
      - no-new-privileges:true      # Block privilege escalation
    read_only: true                 # Immutable filesystem

    # Resource limits (prevent runaway processes)
    deploy:
      resources:
        limits:
          cpus: '1.0'
          memory: 512M

    # Log rotation (prevent disk fill)
    logging:
      driver: 'json-file'
      options:
        max-size: '10m'
```

```
24          max-file: '3'
```

Key Compose commands

`docker compose up -d -build`	Build image and start container in background.
`docker compose down`	Stop and remove container.
`docker compose logs -f`	Stream container logs (like `tail -f`).
`docker compose ps`	List running services and their status.

2.7 Nginx

What it is: A high-performance web server. In this project, it serves the compiled static files (HTML, CSS, JS) that React produced.

Why not just use Node.js to serve? Node.js is single-threaded and designed for dynamic applications. Nginx is purpose-built for serving static files — it uses ∼10 MB of RAM vs. 100+ MB for Node, handles thousands of concurrent connections efficiently, and includes built-in caching, compression, and security features.

The project's `nginx.conf` configures:

Feature	What it does
Rate limiting	Limits requests per IP (10/sec general, 30/sec static) to prevent abuse.
Gzip compression	Compresses responses, reducing bandwidth by 60–80%.
Security headers	`X-Frame-Options`, `CSP`, `X-Content-Type-Options` — defense against XSS, clickjacking, MIME sniffing.
Static asset caching	Files with hashed names get `Cache-Control: max-age=1y, immutable`.
SPA routing	`try_files $uri /index.html` — sends all routes to React so client-side routing works.
Exploit blocking	Blocks `wp-admin`, hidden files, and non-standard HTTP methods.

Listing 8: nginx.conf — SPA routing (the critical part)

```
1 location / {
2     # If the exact file exists, serve it.
3     # Otherwise, serve index.html (React handles routing).
4     try_files $uri $uri/ /index.html;
5 }
```

2.8 Traefik (Reverse Proxy)

What it is: A reverse proxy and load balancer that routes incoming HTTP requests to the correct backend service based on rules (hostname, URL path, headers).

Why it matters: Your Hydra container runs at `hydra.newpaltz.edu/students/you/`. Traefik is the system that looks at the URL, strips the `/students/you/mysite/` prefix, and forwards the

request to your Nginx container on port 80.

```
Browser: GET hydra.newpaltz.edu/students/you/mysite/
                              |
                       Traefik matches PathPrefix
                       Strips /students/you/mysite
                              |
                       Forwards: GET /
                              |
                       Your Nginx container (port 80)
                       Serves index.html
```

In the Hydra cluster, Traefik handles:

- TLS termination (HTTPS)
- Authentication (forward-auth to verify your session)
- Path-based routing (each student gets their own URL namespace)
- Strip-prefix middleware (removes the subpath before forwarding)

You don't need to configure Traefik yourself — the Hydra dashboard handles it when you add a custom endpoint.

3 Project Architecture

3.1 Project Structure

Listing 9: File tree

```
PersonalSite/
  Dockerfile              # Multi-stage build recipe
  docker-compose.yml      # Container orchestration config
  nginx.conf              # Web server configuration
  package.json            # Dependencies and scripts
  tsconfig.json           # TypeScript compiler config
  public/
    index.html            # HTML shell (React mounts into <div id="root">)
    Resume.pdf            # Your resume (shown in PDF viewer)
    icons/favicon.svg     # Browser tab icon
  src/
    index.tsx             # Entry point - mounts <App /> into the DOM
    App/
      App.tsx             # Root component - assembles the page
      App.scss            # Global styles (flexbox layout)
      config.tsx          # YOUR DATA: name, title, links  <-- EDIT THIS
      AppContext.tsx       # Theme state (dark/light mode)
    components/
      Content.tsx         # Renders your name and title
      Buttons.tsx         # Social link buttons (GitHub, etc.)
      Toggle.tsx          # Dark/light mode switch
      Footer.tsx          # "Designed and built by..."
      Particles.tsx       # Animated particle background
```

```
        PdfViewer.tsx      # Resume modal viewer
      constants/
        contact.ts         # Your email address              <-- EDIT THIS
      appearance/
        themes.ts          # Dark and light color palettes
        options.ts         # Particle animation config
      types/
        config.interface.ts  # TypeScript type definitions
```

3.2 How the Pieces Fit Together

1. **You write** React components in TypeScript (`src/`).
2. **npm run build** compiles everything into optimized static files (`build/` folder): minified JS bundles, CSS, and `index.html`.
3. **Docker** packages the build output with Nginx into a self-contained image.
4. **Nginx** serves the static files, handles compression, caching, and security headers.
5. **Traefik** routes requests from `hydra.newpaltz.edu/students/you/mysite/` to your container.

The key insight: **React, TypeScript, Node.js, and npm are only needed at build time.** The running container is pure Nginx serving static files — no JavaScript runtime, no build tools, no `node_modules`.

4 Workshop: Deploy Your Site

Follow these steps in order. Each step builds on the previous one.

4.1 Step 1: Connect to Your Container

Open your terminal and SSH into your Hydra container:

```
$ ssh -i ~/.ssh/hydra_key your_username@hydra.newpaltz.edu -p 2222
```

Or use VSCode in your browser at:
`https://hydra.newpaltz.edu/students/`*`your_username`*`/vscode/`

4.2 Step 2: Clone the Repository

```
$ cd ~
$ git clone https://github.com/ndg8743/PersonalSite.git
$ cd PersonalSite
```

Verify the structure:

```
$ ls
Dockerfile  docker-compose.yml  nginx.conf  package.json
```

```
public/     src/                tsconfig.json
```

4.3 Step 3: Personalize Your Site

You need to edit a few files to make the site yours. Open them in VSCode or use a terminal editor (`nano`, `vim`).

4.3.1 3a. Your name, title, and links

Edit `src/App/config.tsx`:

Listing 10: src/App/config.tsx — change the highlighted values

```
export const config: Config = {
  buttons: [
    {
      // ...
      href: 'https://github.com/YOUR_USERNAME/',  // Your GitHub
    },
    {
      // ...
      href: 'https://linkedin.com/in/YOUR_NAME/', // Your LinkedIn
    },
    // ...
  ],
  name: {
    display: 'Your Full Name',      // <-- YOUR NAME
  },
  title: {
    display: 'Computer Science',    // <-- YOUR TITLE
  },
};
```

4.3.2 3b. Your email

Edit `src/constants/contact.ts`:

```
export const CONTACT = {
  EMAIL: 'your_email@newpaltz.edu',  // <-- YOUR EMAIL
};
```

4.3.3 3c. HTML metadata

Edit `public/index.html` — update the `<title>`, author, and description meta tags. Search for `CHANGE THIS` comments.

4.3.4 3d. Footer credit

Edit `src/components/Footer.tsx` — change the name and URL in the footer link.

4.3.5 3e. Your resume (optional)

Replace `public/Resume.pdf` with your own resume PDF. Keep the same filename.

4.3.6 3f. Set relative asset paths

This is critical for subpath deployment. Add a `homepage` field to `package.json`:

Listing 11: package.json — add this line at the top level

```
{
  "homepage": ".",
  "private": true,
  "scripts": { ... }
}
```

Why `"homepage": "."`?

Without this, React generates absolute paths like `/static/js/main.js`. When your site is served at `/students/you/mysite/`, the browser would request `hydra.newpaltz.edu/static/js/main.js` — which doesn't exist. Setting `homepage` to `"."` makes all paths relative, so assets load correctly from any subpath.

4.4 Step 4: Build the Docker Image

Now build the Docker image. This runs the multi-stage Dockerfile: Stage 1 installs dependencies and compiles React, Stage 2 copies the output into a minimal Nginx image.

```
$ docker build -t my-site .
```

What's happening?

1. Docker pulls `node:18-alpine` (if not cached).
2. Copies `package*.json` and runs `npm ci` (installs dependencies).
3. Copies all source files and runs `npm run build` (compiles React).
4. Docker pulls `nginx:alpine`.
5. Copies the `build/` output and `nginx.conf` into the Nginx image.
6. Tags the final image as `my-site`.

The first build takes 1–3 minutes. Subsequent builds are faster due to layer caching.

Verify the image was created:

```
$ docker images | grep my-site
my-site   latest   abc123def   10 seconds ago   45MB
```

Note the size: ~45 MB. That's because the final image is just Alpine Linux + Nginx + your compiled HTML/CSS/JS. No Node.js, no `node_modules` (which would be 300+ MB).

4.5 Step 5: Run the Container

Start the container, mapping port 3000 on your student pod to port 80 inside the container:

```
$ docker run -d --name my-site -p 3000:80 my-site
```

Flag	Meaning
`-d`	Detached mode (runs in background).
`-name my-site`	Names the container for easy reference.
`-p 3000:80`	Maps host port 3000 → container port 80.
`my-site`	The image name (from Step 4).

Verify it's running:

```
$ docker ps
CONTAINER ID  IMAGE    STATUS         PORTS
a1b2c3d4e5f6  my-site  Up 5 seconds   0.0.0.0:3000->80/tcp

$ curl -s http://localhost:3000 | head -5
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    ...
```

Your site is now running inside a Docker container. But it's only accessible from inside your student pod. The next step makes it public.

4.6 Step 6: Expose Your Site to the Internet

Go to the Hydra dashboard at:
`https://hydra.newpaltz.edu/dashboard/`

1. Find the **Port Routes** section (or "Add Endpoint" button).
2. Enter:
   - **Endpoint name:** `mysite`
   - **Port:** `3000`
3. Click **Add**.

This tells Traefik to route requests from
`https://hydra.newpaltz.edu/students/`*`your_username`*`/mysite/`
to port 3000 on your student pod, where your Docker container is listening.

Test it!

Open your browser and visit:
`https://hydra.newpaltz.edu/students/`*`your_username`*`/mysite/`

You should see your personalized portfolio site with animated particles, your name, title, and social links.

4.7 Step 7: Make It Persistent with Supervisord

Right now, if your student container restarts, the Docker container stops. To make it start automatically, create a supervisord config:

```
$ mkdir -p ~/supervisor.d
$ cat > ~/supervisor.d/mysite.conf << 'EOF'
[program:mysite]
command=docker start -a my-site
autostart=true
autorestart=true
stderr_logfile=/home/student/mysite.err.log
stdout_logfile=/home/student/mysite.out.log
EOF
```

Then reload supervisord:

```
$ supervisorctl reread
$ supervisorctl update
```

What is supervisord?

Supervisord is a process manager that keeps your services running. It automatically restarts them if they crash and starts them when your container boots. Each `.conf` file in `~/supervisor.d/` defines a managed service.

5 Common Operations

5.1 Updating Your Site

After editing source files:

```
$ cd ~/PersonalSite
$ docker stop my-site && docker rm my-site
$ docker build -t my-site .
$ docker run -d --name my-site -p 3000:80 my-site
```

Or as a one-liner:

```
$ docker stop my-site; docker rm my-site; \
  docker build -t my-site . && \
```

```
docker run -d --name my-site -p 3000:80 my-site
```

5.2 Viewing Logs

```
$ docker logs my-site           # All logs
$ docker logs -f my-site        # Stream logs (Ctrl+C to stop)
$ docker logs --tail 20 my-site # Last 20 lines
```

5.3 Debugging

```
# Open a shell inside the running container
$ docker exec -it my-site sh

# Check what Nginx is serving
$ ls /usr/share/nginx/html/

# Check Nginx config
$ nginx -t

# Exit the shell
$ exit
```

5.4 Removing Your Site

```
$ docker stop my-site && docker rm my-site
$ docker rmi my-site
```

Then remove the endpoint from the Hydra dashboard.

6 Key Concepts Recap

Technology	Role	Key File(s)
React	UI framework	`src/App/App.tsx`, `src/components/*.tsx`
TypeScript	Type safety	`tsconfig.json`, all `.tsx` files
SCSS	Global styles	`src/App/App.scss`
styled-components	Theme-aware CSS	Inside component files
Node.js / npm	Build toolchain	`package.json`, `package-lock.json`
Docker	Containerization	`Dockerfile`, `.dockerignore`
Docker Compose	Orchestration	`docker-compose.yml`
Nginx	Web server	`nginx.conf`
Traefik	Reverse proxy	Managed by Hydra (automatic)
supervisord	Process manager	`~/supervisor.d/mysite.conf`

7 Appendix: The Full Request Lifecycle

When someone visits your site, here is exactly what happens:

1. **DNS resolution:** Browser resolves `hydra.newpaltz.edu` to `192.168.1.160` (Hydra's IP).
2. **TLS handshake:** HTTPS connection established (Traefik terminates TLS).
3. **Traefik routing:** URL path `/students/you/mysite/` matches an IngressRoute rule. Traefik authenticates via forward-auth, then strips the prefix.
4. **Service routing:** Traefik forwards the bare `GET /` to the Kubernetes Service `student-you` on port 3000.
5. **Pod networking:** The Service routes to port 3000 on your student pod. Docker's port mapping forwards 3000 → 80 inside your Nginx container.
6. **Nginx serves:** Nginx receives `GET /`, finds `index.html` via `try_files`, adds security headers and compression, and sends the response.
7. **Browser renders:** The browser receives `index.html`, which references `./static/js/main.js` (relative path thanks to `homepage: "."`). It fetches the JS bundle.
8. **React hydrates:** The JavaScript executes, React mounts the component tree, particles animate, and your site is live.

Total time from click to rendered page: typically under 200 ms.

8 Appendix: Troubleshooting

Problem	Solution
`docker: command not found`	Docker runs via DinD sidecar. Check that the sidecar is running: `docker info`. If it fails, restart your container from the dashboard.
Build fails at `npm ci`	Check `package-lock.json` exists. Run `git checkout package-lock.json` to restore it.
Site loads but no styles/scripts	You forgot `"homepage": "."` in `package.json`. Add it and rebuild.
`port is already allocated`	Another container is using port 3000. Run `docker ps` to find it, then `docker stop <name>`. Or use a different port.
Dashboard says "Endpoint already exists"	You already have a route named `mysite`. Delete it first, then re-add.
Blank page after loading	Open browser dev tools (F12) → Console. Look for 404 errors on `.js` files. Usually means the `homepage` fix is missing.

9 Appendix: Going Further

Now that you've deployed a site, here are some challenges:

1. **Add a new page.** React Router can add multi-page navigation (e.g., `/projects`, `/blog`).
2. **Custom domain.** Buy a domain, point its DNS to Hydra, and update the Traefik labels in `docker-compose.yml`.
3. **Add a backend.** Create an Express.js API server as a second service in Docker Compose.
4. **CI/CD.** The project includes a GitHub Actions workflow (`.github/workflows/ci.yml`) that runs tests on every push. Extend it to auto-deploy.
5. **Change the theme.** Edit `src/appearance/themes.ts` to customize colors. Edit `src/appearance/options.` to change particle behavior.